

Reflection Paper

1. What I Implemented

I reproduced the Lesson 5 preprocessing pipeline to prepare the loan dataset for classification. This included handling missing values, encoding categorical features (such as applicant employment status and property area) into numeric form, and scaling the numeric features so that no single variable would dominate the distance- or gradient-based calculations used by the models. I then split the cleaned data into training and test sets using the same random state as in class, so that all three models were trained and evaluated on identical data splits.

Using this shared pipeline, I trained three classifiers to predict loan approval (Approved = 1, Rejected = 0):

- Logistic Regression — the baseline linear model covered in class.
- Random Forest — the ensemble tree-based model covered in class.
- Decision Tree — my additional (third) algorithm, chosen as a simpler, single-tree counterpart to Random Forest.

Each model was trained on the same features and evaluated with the same metrics (Accuracy, Precision, Recall, and F1-Score), plus a confusion matrix, so the results could be compared fairly.

2. Comparison of Models

For the sanity check, I picked a single test case with a known actual label of Approved (1) and asked all three trained models to predict it. All three models correctly predicted “Approved” for this case, so on this individual example the models agreed and none of them produced an unrealistic or contradictory prediction.

Where the models diverged was in their overall metrics and confusion matrices rather than in this single sanity check. Logistic Regression and Random Forest produced identical confusion matrices (11 true positives, 2 false negatives, 4 false positives, 3 true negatives), giving both an accuracy of 0.700 and an F1-score of 0.786. The Decision Tree behaved differently: it predicted every actual approval correctly (13 true positives, 0 false negatives), giving it a perfect recall of 1.000, but it also produced more false positives (5), which pulled its precision down slightly to 0.722.

In terms of realism, I'd consider Logistic Regression and Random Forest more realistic overall, because a real loan-approval model that never misses a single approval (recall = 1.000) and simultaneously has the lowest precision of the three is a sign of overfitting to the “Approved” class rather than genuinely strong performance. Logistic Regression's more balanced precision and recall (0.733 vs. 0.846) suggest it is making a more even trade-off between approving good applicants and rejecting risky ones, which better reflects how a lender would actually want the model to behave.

3. Understanding Random Forest

a Random Forest is an ensemble learning method that builds many individual decision trees and combines their outputs to make a single, more reliable prediction. Each tree in the forest is trained on a random subset of the training data (sampled with replacement, known as bagging) and considers only a random subset of features at each split. This randomness means that no two trees in the forest are identical, and each one ends up capturing slightly different patterns in the data.

For classification specifically, every tree in the forest independently predicts a class label (in this case, Approved or Rejected) for a given input. The Random Forest then takes a majority vote across all the trees' predictions and outputs whichever class received the most votes. Because the errors of individual trees tend to be somewhat random and uncorrelated, averaging over many trees cancels out a lot of that noise. This is why Random Forest typically generalizes better and overfits less than a single decision tree, even though each individual tree in the forest may itself be prone to overfitting.

4. Other Algorithms (My Third Classifier)

Choice: I chose the Decision Tree classifier as my third algorithm. I chose it specifically because it is the building block that Random Forest is made of — training a single, standalone Decision Tree on the same data let me see directly what one tree does on its own versus what happens when many trees are combined and voted over in a Random Forest.

From my research, a Decision Tree works by repeatedly splitting the training data into smaller and smaller subsets based on feature values, choosing at each step the split that best separates the classes (commonly measured using Gini impurity or information gain/entropy). This produces a tree of if/then decision rules that can be followed from the root down to a leaf, where the leaf's majority class becomes the prediction.

Advantage: Decision Trees are highly interpretable — you can literally trace the path of splits that led to a particular loan decision, which is valuable in a domain like lending where explainability matters.

Limitation: a single Decision Tree is prone to overfitting, especially when it is allowed to grow deep; it can memorize quirks of the training data (including noise) rather than learning generalizable patterns, which is reflected in it producing more false positives than the other two models here.

Compared to Logistic Regression and Random Forest, the Decision Tree in my results had the highest accuracy (0.750) and by far the highest recall (1.000), meaning it caught every actual approval in the test set. However, its precision (0.722) was the lowest of the three, meaning it also mislabeled more rejected applicants as approved. Logistic Regression and Random Forest were identical to each other on every metric, both trading a bit of recall for better precision than the Decision Tree.

5. Metrics Discussion

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	0.700	0.733	0.846	0.786
Random Forest	0.700	0.733	0.846	0.786
Decision Tree	0.750	0.722	1.000	0.839

Looking across the four metrics: the Decision Tree had the best Accuracy (0.750) and the best Recall (1.000, a perfect score). Logistic Regression and Random Forest tied for the best Precision (0.733), and the Decision Tree had the best F1-Score (0.839), narrowly ahead of Logistic Regression and Random Forest's tied 0.786.

This pattern tells me that each model has a different strength-and-weakness profile rather than one model being strictly better. The Decision Tree's perfect recall means it is very good at not missing approvals, but this comes at the cost of precision — it is more willing to falsely approve applicants who should have been rejected, which is exactly the overfitting risk that single trees are known for. Logistic Regression and Random Forest are more conservative and balanced: they miss a few more approvals (lower recall) but are more precise when they do label someone as approved. In a lending context, this trade-off matters a lot, because false positives (approving someone who shouldn't be approved) carry real financial risk, while false negatives (rejecting someone who should have been approved) are a missed business opportunity but a smaller risk.

6. My Findings

Based on these results, I would use Logistic Regression (or, equivalently here, Random Forest, since they produced identical metrics on this dataset) for loan approval prediction rather than the Decision Tree. Even though the Decision Tree edged out the others on accuracy and F1-score, its perfect recall combined with the lowest precision of the three models is a red flag for overfitting on a small test set, not genuine superior performance. A model that approves too easily is risky in a real lending scenario, since the cost of a bad approval (default risk) is generally higher than the cost of a cautious rejection.

Logistic Regression also has the practical advantage of being simple, fast to train, and easy to interpret through its coefficients, which matters in a regulated domain like lending where decisions may need to be explained to applicants or auditors. Random Forest would be my second choice if the dataset were larger and more complex, since its ensemble structure would likely become more valuable at scale (more data to build diverse trees) and would guard against the overfitting that hurt the single Decision Tree here. Overall, I would prioritize the model with the more balanced precision/recall trade-off and greater interpretability over the model with the single best headline number.